Capitolul 3

Reprezentarea numerelor si aritmetici

Una dintre cele mai importante caracteristici, care determina disponibilitatea procesoarelor DSP pentru o aplicatie anume, este tipul de reprezentare numerica binara folosita de procesor. Cele mai folosite reprezentari binare a datelor numerice de procesoarele DSP comerciale sunt reprezentate in Fig. 3.1.

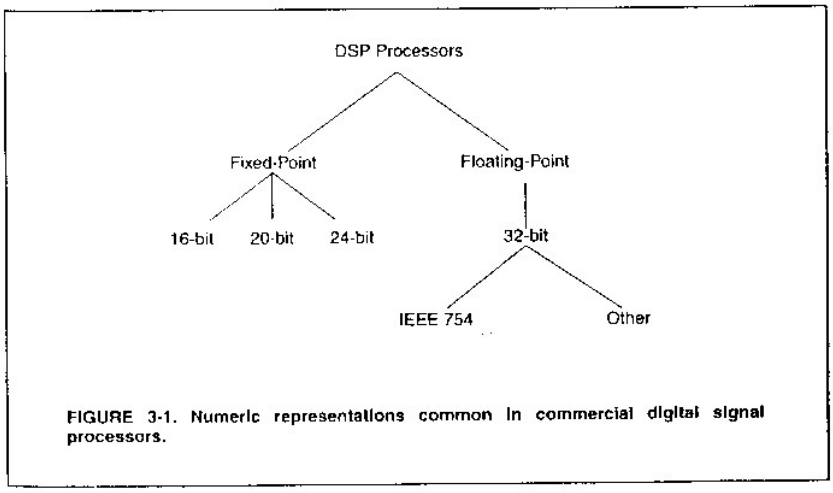
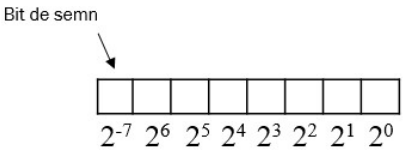


FIGURE 3-1. Numeric representations common in commercial digital signal processors.

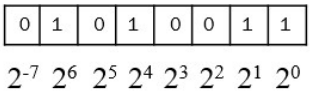
3.1 Virgula fixa si virgula mobila

Primele procesoare DSP au folosit aritmetica in *virgula fixa*, si de fapt procesoarele DSP cu virgula fixa inca sunt dominante. In procesoarele cu virgula fixa, **numerele sunt reprezentate fie ca intregi** (aritmetica intreaga), fie ca **fractii cu valoarea intre –1,0 si +1,0** (aritmetica fractionara). Algoritmii si hardware-ul utilizat pentru a implementa aritmetica fractionara sunt virtual identici cu aceia folositi pentru aritmetica cu intregi. Principala diferenta intre cele doua reprezentari este legata de modul in care sunt manipulate rezultatele operatiilor de multiplicare. In practica, majoritatea procesoarelor DSP cu virgula fixa suporta atit aritmetica fractionara cit si cea cu intregi. Prima este mai folositoare pentru algoritmii de procesare a semnalului, in timp ce a doua este utilizata pentru operatii de control, calculul adreselor si pentru alte operatii care nu au legatura cu semnalele. Figurile 3.2 si 3.3 ilustreaza reprezentari simple ale intregilor si fractiilor. Se observa ca folosind aritmetica cu intregi pe un DSP cu virgula fixa uneori calculele consuma mai mult timp sau sunt supuse unor constringeri decit atunci cind este folosita aritmetica fractionara.

Exemplu de calcul in virgula fixa cu aritmetica intreaga

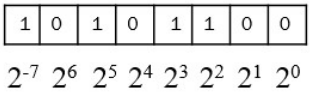


Exemplul 1:



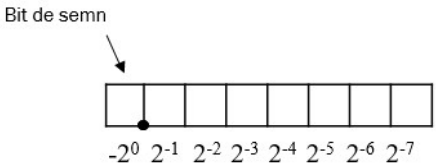
$= 2^6 + 2^4 + 2^1 + 2^0 = 64 + 16 + 2 + 1 = 83$

Exemplul 2:

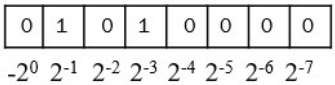


$= 2^{-7} + 2^5 + 2^3 + 2^2 = -128 + 32 + 8 + 4 = -84$

Exemplu de calcul in virgula fixa cu aritmetica fractionara

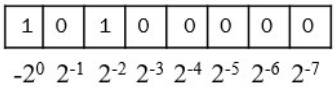


Exemplul 1:



$= 2^{-1} + 2^{-3} = 0.5 + 0.125 = 0.625$

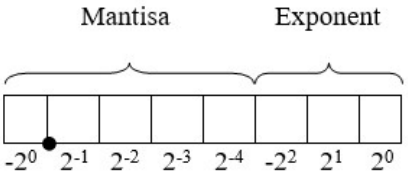
Exemplul 2:



$= -2^0 + 2^{-2} + 2^{-4} = -1.0 + 0.25 + 0.0625 = -0.6875$

Alta clasa de procesoare DSP foloseste aritmetica cu *virgula mobila*, unde numerele sunt reprezentate prin *mantisa* si *exponent*. Mantisa este de obicei valoarea fractionara a semnalului. Exponentul este un intreg care reprezinta numarul de ranguri binare cu care virgula mantisei (similar cu virgula intr-un numar in baza 10) trebuie deplasata la stinga sau la dreapta pentru a obtine valoarea numarului original. Valoarea zecimala reprezentata este data de relatia:

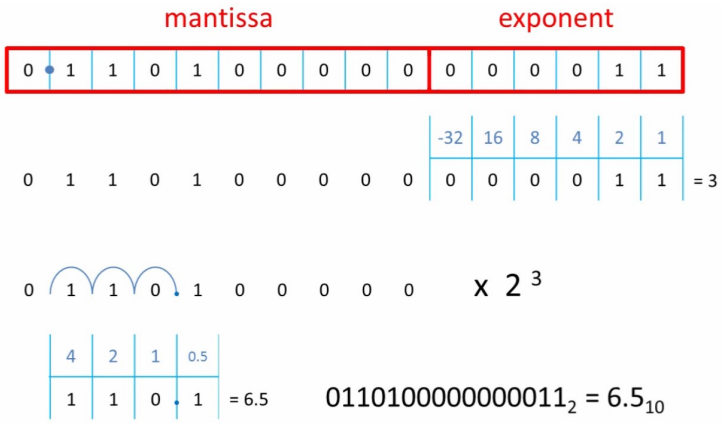
valoare = mantisa x $2^{exponent}$



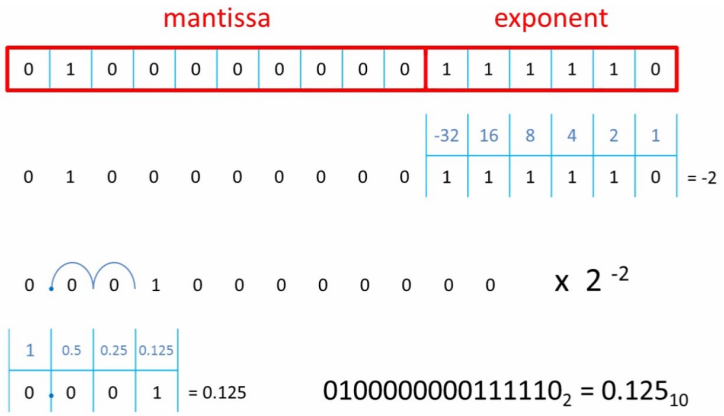
In general, procesoarele cu virgula mobila accepta si date cu virgula fixa (de obicei intregi). Aceasta este necesara pentru a facilita operatiile cu date care sunt inerent intregi, cum ar fi adrese ale memoriei.

Aritmetica cu virgula mobila este un mecanism mai flexibil si mai general decit cea cu virgula fixa. Cu virgula mobila, utilizatorii sistemului au acces la o gama dinamica mai larga (raportul intre cel mai mare si cel mai mic numar care poate fi reprezentat) si in multe cazuri la o precizie mai buna.

Exemplu de conversie in virgula mobila a unui numar pozitiv cu Mantisa pe 10 biti si Exponent pozitiv pe 6 biti:



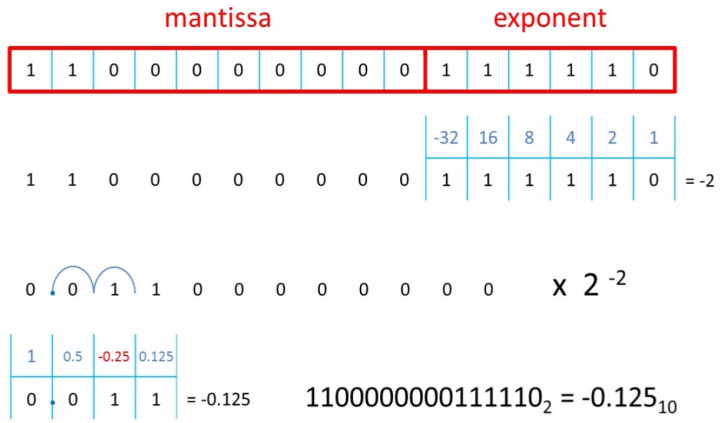
Exemplu de conversie in virgula mobila a unui numar pozitiv cu Mantisa pe 10 biti si Exponent negativ pe 6 biti:



Exemplu de conversie in virgula mobila a unui numar negativ cu Mantisa pe 10 biti si Exponent pozitiv pe 6 biti:



Exemplu de conversie in virgula mobila a unui numar negativ cu Mantisa pe 10 biti si Exponent negativ pe 6 biti:



- **Precizia**. Definitia data preciziei este bazata pe ideea erorii de cuantizare. Eroarea de cuantizare este eroarea numerica introdusa atunci cind un format de numar mai lung (cu mai multi biti) este convertit la unul mai scurt. De exemplu, in cazul numerelor zecimale, cind rotunjim valoarea 1,325 la 1,33, introducem o eroare de cuantizare de 0,005. Cu cit eroarea de cuantizare posibila este mai mare fata de marimea valorii reprezentate, cu atit precizia este mai mica.

Pentru un format cu virgula fixa, definim precizia maxima posibila ca fiind egala cu numarul de biti din format. De exemplu, un format fractionar pe 16 biti are o precizie maxima de 16 biti. Aceasta definitie este bazata pe calculul raportului dintre marimea valorii reprezentate si marimea erorii maxime de cuantizare, la convertirea de la o reprezentare mai precisa prin rotunjire.

Formal aceasta poate fi scrisa: *Precizia maxima* [in biti] = log_2 (|*valoarea maxima*|/|*eroarea maxima de cuantizare*|)

Pentru reprezentarea fractionara pe 16 biti, valoarea maxima care poate fi reprezentata este –1,0. Cind convertim aceasta data la un format fractionar pe 16 biti, de la unul mai precis, prin rotunjire, eroarea maxima de cuantizare este 2^{-16}. Folosind relatia de mai sus, putem calcula precizia maxima pentru acest format, respectiv $\log_2 (1/2^{-16})$, sau 16 biti, deci aceeasi cu lungimea totala a formatului.
Se observa ca, daca data care este reprezentata are o marime mai mica decit cea maxima, precizia obtinuta este mai mica decat precizia maxima posibila. Aceasta subliniaza importanta scalarii atente a semnalului cind folosim aritmetica cu virgula fixa. Scalarea este folosita pentru a mentine precizia dorita, prin modificarea intervalului de valori ale datelor astfel incat sa fie apropiat de intervalul maxim oferit de reprezentarea numerica folosita. Scalarea este discutata in detaliu in cap. 4," Calea datelor".

Folosind aceeasi definitie pentru formatul cu virgula mobila, precizia maxima posibila este numarul de biti din mantisa. Procesoarele cu virgula mobila mentin o precizie foarte buna fara efort suplimentar din partea programatorului.

In practica, procesorele DSP cu virgula mobila folosesc in general un format pe 32 de biti, cu o mantisa pe 24 de biti si un bit intreg, care furnizeaza o precizie de 16 biti. Deci, in timp ce in teorie alegerea intre aritmetica in virgula fixa si in virgula mobila poate fi independenta de alegerea preciziei, in practica procesorul cu virgula mobila, asigura o precizie mai inalta.

- **Gama dinamica.** Asa cum am mentionat, gama dinamica este definita ca raportul dintre cel mai mare si cel mai mic numar reprezentabil intr-un anumit format de data. In aceasta privinta formatele cu virgula mobila au avantajul major. De exemplu, consideram

reprezentarea fractionara in virgula fixa pe 32 de biti. Valoarea minima care poate fi reprezentata de acest format este 2^{-31}; iar valoarea maxima este $1,0 - 2^{-31}$. Raportul dintre acestea, care este gama dinamica a acestui format de data, este aproxmativ $2,15 \times 10^{9}$, sau aproximativ 187 decibeli (dB). Un format in virgula mobila tot cu 32 de biti (cu o mantisa de 24 de biti si un exponent pe 8 biti) poate reprezenta numere de la aproximativ $5,88 \times 10^{-39}$ la $3,40 \times 10^{38}$, acoperind o gama dinamica de aproximativ $5,79 \times 10^{76}$, sau peste 1535 dB. Deci, folosind acelasi numar de biti, formatul cu virgula mobila asigura o gama dinamica mult mai mare.

In aplicatii, gama dinamica se traduce intr-un interval de amplitudini ale semnalului care pot fi prelucrate, mentinind o fidelitate suficient de mare. Aplicatii diferite au nevoie de game dinamice diferite. Pentru aplicatii in telecomunicatii o gama dinamica de 50 dB este in general suficienta. Pentru aplicatii audio de fidelitate inalta este nevoie de un domeniu de 90 dB. Adesea este utila o gama dinamica mai mare decit cerintele minime ale aplicatiei, daca reprezentarea numerica si hardware-ul aritmetic al procesorului o permit, aceasta degrevind programatorul de o parte din problemele de scalare care pot sa apara. Scalarea este discutata in capitolul 4 - “Calea de date“.

Procesoarele DSP cu virgula mobila sunt, in general, mai scumpe decit cele cu virgula fixa, dar mai usor de programat. Costul ridicat este datorat circuitului mai complex necesar procesorului cu virgula mobila, care implica un cip mai mare. In plus, lungimea mai mare a cuvintelor procesoarelor cu virgula mobila necesita cresterea dimensiunii magistralelelor externe si a memoriilor externe, marind costul total al sistemului.

Avantajul folosirii mai simple a procesoarelor cu virgula mobila este datorat faptului ca in multe cazuri programatorul este degrevat de sarcina controlului gamei dinamice si preciziei. Din contra, la un procesor cu vigula fixa, programatorul trebuie sa fie atent adesea la scalarea datelor in diverse stadii ale programului pentru a asigura o performanta numerica echivalenta cu gama dinamica si precizia limitata a procesorului cu virgula fixa. Cu toate ca reprezentarea in virgula mobila ofera performante superioare, trebuie mentionat ca, din cauza dificultatilor de implementare, potentialul acestei reprezentari nu este deplin valorificat. Doar prin folosirea unor arhitecturi complet paralele combinate cu logica combinationala pipeline, pentru obtinerea unei viteze mari de prelucrare, va fi valorificat potentialul acestei reprezentari.

Cea mai mare parte a aplicatiilor dedicate folosesc procesoare cu virgula fixa pentru ca prioritatea o reprezinta costul scazut. Programatorii si proiectantii de algoritmi stabilesc gama dinamica si precizia necesara in aplicatia lor, fie analitic, fie prin simulare, si apoi adauga operatii de scalare in cod daca este necesar. Pentru aplicatii care sunt mai putin sensibile la cost, sau care au cerinte stricte pentru gama dinamica si de precizie, sau unde programarea mai usoara este mai importanta, sunt mai potrivite procesoarele cu virgula mobila.

3.2 Largimea nativa a cuvintelor pentru date

Lungimea nativa a cuvintelor pentru reprezentarea datelor intr-un procesor DSP este lungimea datelor pe care magistralele procesorului si calea datelor o pot manipula intr-un singur ciclu instructiune. Toate DSP-urile cu virgula mobila uzuale folosesc o lungime nativa de 32 biti. Pentru procesoarele cu virgula fixa cea mai folosita lungime este 16 biti. Familiile de procesoare DSP5600x si DSP563xx (Motorola), folosesc cuvinte pe 24 biti, in timp ce procesorul ZR3800x (Zoran), foloseste cuvinte de lungime 20 biti. Marimea cuvintelor are un impact major asupra costului procesorului pentru ca influenteaza marimea cipului si numarul de pini ai capsulei (package pins) precum si marimea si numarul de dispozitive de memorie externa conectate la procesor. De aceea, proiectantii incearca sa foloseasca cipuri cu cea mai mica lungime a cuvintelor pe care o poate accepta aplicatia respectiva. Nucleul DSP - CD2450 (Clarkspur Design) este unic in privinta posibilitatilor proiectantului cipului de a alege orice marime a cuvantului intre 16 si 24 de biti.

In ceea ce priveste alegerea intre procesoarele cu virgula fixa si cele cu virgula mobila, exista adesea un compromis (trade-off) intre marimea cuvantului si cresterea complexitatii. O aplicatie care pare sa necesite date pe 24 biti pentru o performanta adecvata, poate fi uneori obtinuta cu un procesor pe 16 biti cu pretul folosirii unor algoritmi mai complecsi si/sau programare mai complexa.

3.3 Precizia extinsa

Precizia extinsa inseamna folosirea reprezentarilor pentru date care asigura o precizie mai mare decat aceea oferita de formatul de date nativ procesorului. Precizia extinsa poate fi obtinuta in doua moduri.

In primul rind, multe procesoare cu virgula fixa si cu virgula mobila asigura suport incorporat pentru un format cu precizie extinsa pentru operatii care au loc exclusiv in interiorul caii de date a procesorului (vezi cap.4 pentru o discutie asupra cailor datelor). Aceasta inseamna ca atata timp cat o serie de operatii aritmetice sunt realizate exclusiv in interiorul caii de date si nu implica transferul de date intermediare la si de la memorie, este posibila o marime a cuvantului mai mare decat marimea cuvantului nativ pentru date. Aceasta permite ca o serie de operatii aritmetice sa fie executate folosind precizie extinsa si/sau o gama dinamica mai mare, cu o operatie finala de rotunjire executata cand rezultatul este inmagazinat in memorie.

In al doilea rind, este posibil de a obtine o precizie aritmetice extinsa prin construirea unor cuvinte mai mari din secvente de cuvinte de date native. De exemplu, cu un procesor in

virgula fixa pe 16 biti, un programator poate forma cuvinte pe 32 de biti prin legarea unor perechi de cuvinte pe 16 biti. Programatorul poate implementa operatii aritmetice cu precizie extinsa folosind o serie de instructiuni de adunare in precizie simpla.

Desigur, deoarece fiecare operatie aritmetica in precizie extinsa necesita o secventa de instructiuni in precizie simpla, aritmetica cu precizie extinsa este mult mai lenta decat precizia simpla. Unele precesoare asigura suport pentru aritmetica in precizie extinsa. Aceasta include posibilitatea de a pastra bitul de transport (carry bit), rezultat de la o operatie de adunare de precizie simpla, pentru a fi folosit ca intrare intr-o adunare ulterioara, si posibilitatea de a considera operanzii multiplicarii ca fiind numere cu sau fara semn, sub controlul programului.

Daca cea mai mare parte a unei aplicatii poate fi executata cu aritmetica in precizie simpla, dar este necesara o precizie mai mare pentru o mica sectiune a codului, atunci folosirea selectiva a aritmeticii de precizie extinsa are sens. Daca cea mai mare parte a aplicatiei necesita precizie ridicata, atunci un procesor cu o lungime nativa mai mare a datelor ar fi o alegere mai buna, daca este disponibil.

3.4 Emularea virgulei mobile si a virgulei mobile pe bloc

Cand folosim un procesor cu virgula fixa, este posibil sa obtinem precizia si gama dinamica caracteristice aritmeticii de precizie extinsa prin folosirea unor rutine software care emuleaza comportamentul unui procesor cu virgula mobila. Unii producatori de procesoare furnizeaza o biblioteca de rutine de emulare a virgulei mobile pentru procesoarele lor cu virgula fixa. Daca o astfel de biblioteca nu este disponibila, rutinele de emulare trebuiesc scrise de utilizator. Rutinele de virgula mobila sunt in general foarte scumpe de executat in termenii numarului de tacte ale procesorului. Aceasta implica faptul ca emularea virgulei mobile este potrivita doar daca o parte foarte mica a calculelor aritmetice necesare unei aplicatii implica virgula mobila. Daca o parte semnificativa de aritmetica in virgula mobila este necesara, atunci un procesor cu virgula mobila este alegerea mai potrivita.

O cale pentru a obtine o gama dinamica si o precizie crescuta pentru anumite date in implementarea cu procesor cu virgula fixa este o reprezentare cu *virgula mobila pe bloc*. Cu acest tip de reprezentare, un grup de numere cu mantise diferite dar un singur exponent, este tratat ca un bloc de date. In loc ca exponentul sa fie stocat ca o parte a fiecarui cuvant de date, asa cum se face in cazul virgulei mobile de uz general, exponentul comun este inmagazinat intr-un cuvant de data separat. De exemplu, pentru un bloc de opt valori de data au acelasi exponent mantisele pot fi stocate separat, iar exponentul comun ar fi stocat intr-un cuvant de data unic. In acest caz, stocarea intregului bloc de opt valori de data ar necesita noua locatii de memorie (opt pentru mantise si una pentru exponent).

Virgula mobila pe bloc este folosita pentru a mentine o gama dinamica si o precizie ridicata, specifica formatelor aritmetice de virgula mobila native ale procesorului. De exemplu, o rutina pentru un filtru care poate produce o serie de valori de date pe 32 de biti poate fi redusa de programator la o reprezentare cu virgula mobila pe bloc folosind mantise de 16 biti si un exponent de 4 biti pentru fiecare bloc de date. Din prezentarea anterioara asupra formatelor in virgula mobila a rezultat ca valoarea reprezentata printr-un numar in virgula mobila este calculata printr-o expresie de forma:

valoare = mantisa x $2^{exponent}$

Cu acest format, exponentul pentru un bloc de date este in general determinat de elementul datei din bloc cu cea mai mare magnitudine. Daca un anumit bloc de date este constituit in totalitate din valori mici, atunci aceste valori mici pot fi deplasate sapte pozitii la stanga, rezultind un exponent negativ relativ mare. Deplasarea la stanga muta bitii de ordin inferior in pozitii unde vor fi protajati cand largimea datei va fi redusa. Daca un bloc de date contine valori mari ale datei, valorile sunt deplasate la stanga doar cateva pozitii sau chiar nici una, si va rezulta un mic exponent negativ (sau chiar un exponent nul). Daca data nu poate fi deplasata, atunci precizia rezultata este aceeasi ca si cand ar fi fost folosit direct formatul nativ in virgula fixa.

Conversia intre formatul nativ in virgula fixa al procesorului si formatul in virgula mobila pe bloc este executat de programator prin software. Unele procesoare au hardware specializat pentru conversia datelor in acest format. Cea mai cunoscuta este instructiunea de “detectie a exponentului”. Aceasta instructiune calculeaza deplasarea necesara pentru a converti un rezultat intermediar de precizie inalta (de exemplu, o valoarea dintr-un acumulator) in format virgula mobila pe bloc. Instructiunile specializate, suport pentru acest format, sunt discutate in capitolul 7.

Procesoarele familiei TMS320C54 (TI) poseda doua instructiuni support pentru reprezentarea datelor in virgula mobila pe bloc:

- instructiunea EXP pentru calculul exponentului unei date;
- instructiunea NORM separa mantisa de exponent dupa ce a fost executata instructiunea EXP.

3.5 Standardul IEEE 754 pentru reprezentarea in virgula mobila

In 1985, Institutul de inginerie electrica si electronica a lansat standardul IEEE-754 [IEE85], care defineste formatele standard pentru reprezenatarea datelor in virgula mobila si un set de reguli pentru aritmetica in virgula mobila. Regulile specifica de exemplu, algoritmii de rotunjire care ar trebui executati intr-un procesor in virgula mobila si modul in care procesorul

ar trebui sa manipuleze conditiile de exceptie aritmetica, precum impartirea la zero sau depasirea. Standardul specifica doua formate de baza pentru reprezentare:

- *Precizia simpla* pe 32 de biti cu 24 de biti mantisa si 8 biti exponentul;
- *Precizia dubla* pe 64 de biti cu 52 de biti mantisa si 12 biti exponentul

Atit mantisa cit si exponentul sunt reprezentate ca numere cu semn.

Cateva procesoare DSP comerciale asigura suport hardware partial pentru aritmetica si formatele pentru virgula mobila IEEE-754. Procesorul DSP96002 (Motorola) asigura suportul hardware pentru aritmetica in virgula mobila cu precizie simpla asa cum este specificat in IEEE-754. Procesoarele din familia ADSP-210xx (Analog Devices) asigura suport hardware aproape complet pentru aritmetica in virgula mobila cu precizie simpla asa cum se specifica in standard.

Alte procesoare cu virgula mobila, cum ar fi DSP32xx (AT&T), nu respecta standardul IEEE-754, dar asigura hardware special pentru conversia rapida a numerelor intre reprezentarea in virgula mobila din interiorul procesorului si reprezentarea IEEE-754. Suportul hardware pentru conversia formatului poate fi important in aplicatii care cer un DSP care sa nu fie in conformitate cu IEEE-754 pentru a-l interfata cu alte procesoare care folosesc reprezentarea IEEE-754. Fara suport hardware de conversie, DSP-ul in virgula mobila neconform cu standardul, trebuie sa foloseasca rutine software pentru a converti intre diversele formate in virgula mobila, si acesta conversie software poate consuma ceva timp. De aceea, cei care dezvolta aplicatii care necesita un DSP care sa fie interfatat cu alte procesoare care folosesc reprezentarea IEEE-754 ar trebui sa evalueze cu grija eficienta conversiei software, sau sa aleaga un procesor cu capacitati hardware de conversie (sau unul care foloseste intern formatele IEEE in virgula mobila).

3.6 Relatia intre marimea cuvintelor pentru date si a celor pentru instructiuni

In timp ce majoritatea procesoarelor DSP folosesc cuvinte pentru instructiuni de lungime egala cu cea a cuvintelor pentru date, exista si unele care nu fac acest lucru. De exemplu, familia ADSP-21xx (Analog Devices) si MDSP2780 (IBM) folosesc cuvinte de date pe 16 biti si cuvinte instructiune pe 24 biti. In mod similar, ZR3800x (Zoran) de 20 biti Zoran, foloseste cuvinte instructiune pe 32 biti. Procesoarele cu marimi diferite ale cuvintelor iau in general masuri de prevedere pentru a permite datelor sa fie stocate in memoria programului, de exemplu, folosirea a 16 biti dintr-o locatie de memorie a programului de 24 biti. Daca acest aranjament functioneaza, in schimb e clar ca nu este cel mai eficient mod de folosire a memoriei, deoarece o parte importanta din memoria pentru programe, folosita pentru a stoca date, este irosit, si aceasta poate avea impact asupra costului total al sistemului.